

1. El *Buffer Cache* reduce el número de lecturas físicas, pero no de escrituras físicas, sobre los discos montados. F
2. El *Buffer Cache* reduce tanto el nº de lecturas como de escrituras físicas sobre los discos montados. V
3. El *Buffer Cache* permite reducir operaciones de lectura sobre los discos montados, pero no operaciones de escritura en los mismos. F
4. El *Buffer Cache* reduce el nº de lecturas físicas sobre los discos montados, pero no el nº de escrituras físicas. F
5. El *Buffer Cache* reduce el nº de lecturas o escrituras físicas sobre los discos montados. V
6. El *Buffer Cache* minimiza posibles errores, ante caídas de alimentación, cuando se realizan operaciones de cambio de datos/metadatos en el sistema de ficheros. F
7. La librería estándar de C incluye el código binario de *unlink()*. F
8. El código binario de *lseek()* se encuentra en la librería estándar de C. F
9. El código binario de *chmod()* se encuentra en la librería estándar de C. F
10. El código binario de *fork()* se encuentra en la librería estándar de C. F
11. Al ejecutar la función de entrada/salida de C *perfor()*, el proceso está ejecutando exclusivamente en modo usuario. F
12. Al ejecutar la función de entrada/salida de C *printf*, el proceso está ejecutadno exclusivamente en modo sistema. F
13. El número de aperturas de un fichero se mantiene en el inodo en memoria, no en el inodo en disco. V
14. Mientras un fichero está abierto, el S.O. matiene una copia de su inodo en memoria principal. V
15. El S.O. puede abrir varias veces un fichero y con distintos modos de apertura, y ese número de aperturas se mantiene en el inodo en memoria, pero no en el inodo en la lista de inodos en disco. V
16. La asociación entre el uid del inodo y el nombre simbólico del usuario se encuentra en el fichero */etc/passwd* y puede obtenerse con *getpwuid*. V
17. El bit *UID* y el bit pegajoso forman parte del campo de "modo" del inodo. V
18. Un proceso puede abrir varias veces un fichero, pero debe ser con el mismo modo de apertura. F
19. Las librerías dinámicas facilitan la actualización de componentes del SO. V
20. Al lincar un código con una librería dinámica, las funciones necesarias de la librería se integran en el fichero ejecutable. F
21. Las librerías dinámicas de ficheros con asignación indexada de bloques, puede existir fragmentación externa. F

22. Al linkar un código con una librería dinámica, las funciones necesarias de la librería se integran en el fichero ejecutable. F
23. El código linkado con una librería dinámica es "autocontenido". F
24. Al ejecutar la función $\sin(x)$ de la librería matemática, el proceso está ejecutando exclusivamente en modo usuario. V
25. Al ejecutar la función *printf* de C se incrementa tanto el tiempo de ejecución en modo usuario como en modo kernel. V
26. Al ejecutar la función *perrot* de C se incrementa el tiempo de ejecución en modo usuario pero no en modo kernel.
27. Al ejecutar la función *printf* de C se incrementa tanto el tiempo de ejecución en modo usuario como en modo kernel. V
28. Las librerías estáticas permiten compartir código entre procesos en tiempo de ejecución. F
29. Al linkar un código con una librería estática, el código en memoria de las funciones necesarias de la librería se puede compartir entre varios procesos. F
30. Un sistema de ficheros con registro disminuye la fragmentación externa. F
31. La idea fundamental de un sistema de ficheros Unix basado en registro es llevar control/registro de las creaciones y eliminaciones de los ficheros a lo largo del tiempo. F
32. Es posible crear *hard links* entre diferentes sistemas de ficheros montados. F
33. El tipo de fichero "link simbólico" se codifica en el campo "modo" del inodo. V
34. Es posible crear *soft links* entre diferentes sistemas de ficheros montados. V
35. Una llamada al sistema se puede implementar con una interrupción software, que cambia el modo de ejecución del procesador. V
36. Cuando un proceso ejecuta en modo usuario, se puede ejecutar todo el repertorio de instrucciones máquina del procesador. F
37. El S.O. mantiene una copia en memoria principal de un fichero abierto, al menos hasta el último cierre del fichero. V
38. La fragmentación externa no existe con un sistema de ficheros con asignación indexada de los bloques de datos. V
39. El contenido de un directorio se almacena en el área de datos. V
40. *fgetc* es una llamada al sistema operativo. F